

A learning companion on a scarce GPU

An agentic RAG app on our own vLLM cluster on Lambda GPUs. Admission control decides what enters, and the router decides where it goes. vLLM runs the model.

Chetan Anand

2026-10-10

github.com/chetananand/learning-companion-serving

The app: a learning companion over my bookmarks

My Notion database has 998 bookmarks of pages that I want to learn from. The companion answers my questions from these bookmarks, and it cites a source for each fact.

Quick mode: one agent searches the bookmarks, reads a page, and reads a figure with OCR.

Verified mode: bookmarks get old, so a second agent checks up to 3 claims on live web pages. Our code decides when a live fact wins over a bookmark.

In course terms

Track B with a Track A tool: an agent that retrieves.

▶ **Clip 1, 27 s: a verified answer, both claims verified**

I ask a question

in the chat of the app



One agent finds the facts

search the bookmarks, read a page, read a figure



The answer

with a source for each fact

↓ verified mode only

A second agent checks the claims

up to 3 claims, on live web pages



The final answer

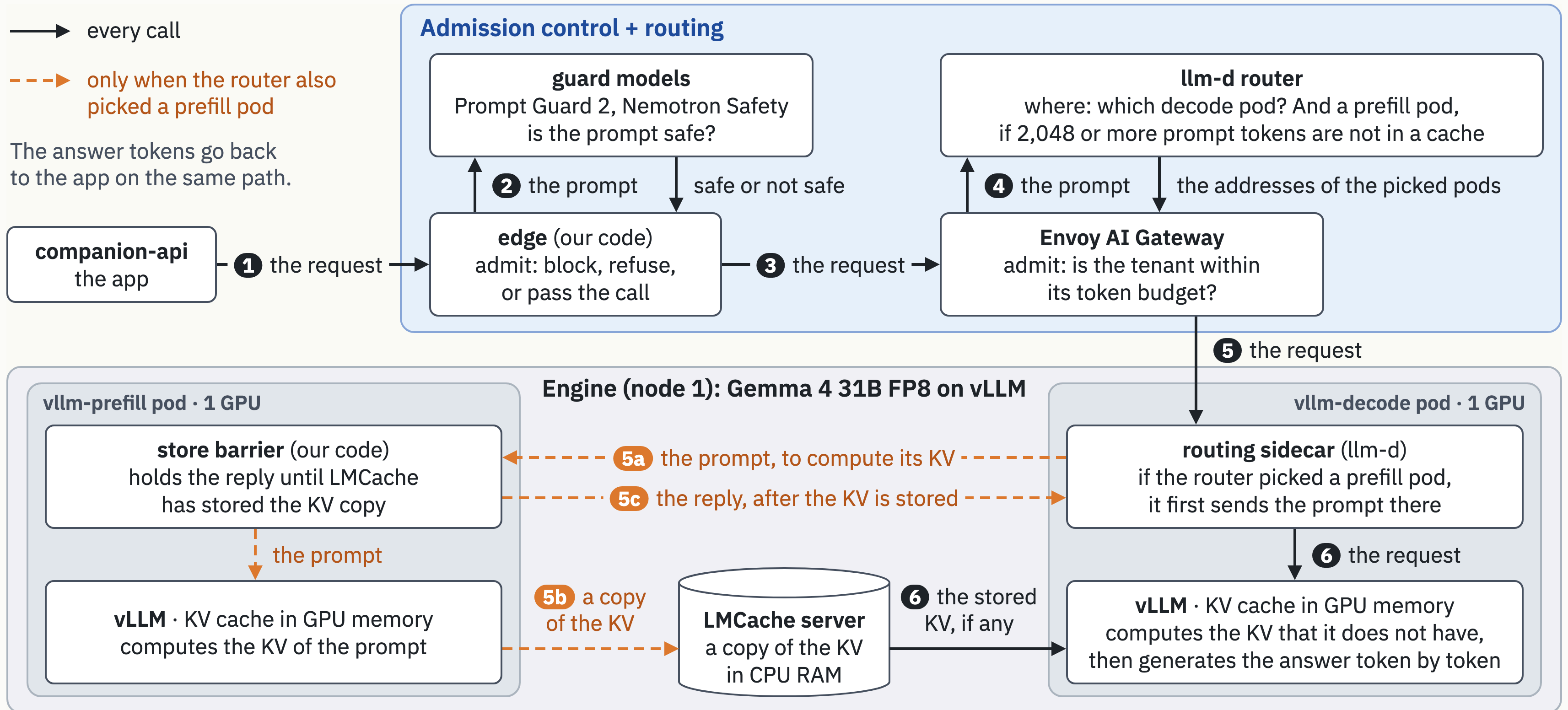
with the status of each claim

The path of one LLM call

→ every call

---> only when the router also picked a prefill pod

The answer tokens go back to the app on the same path.



The models, and the job of each

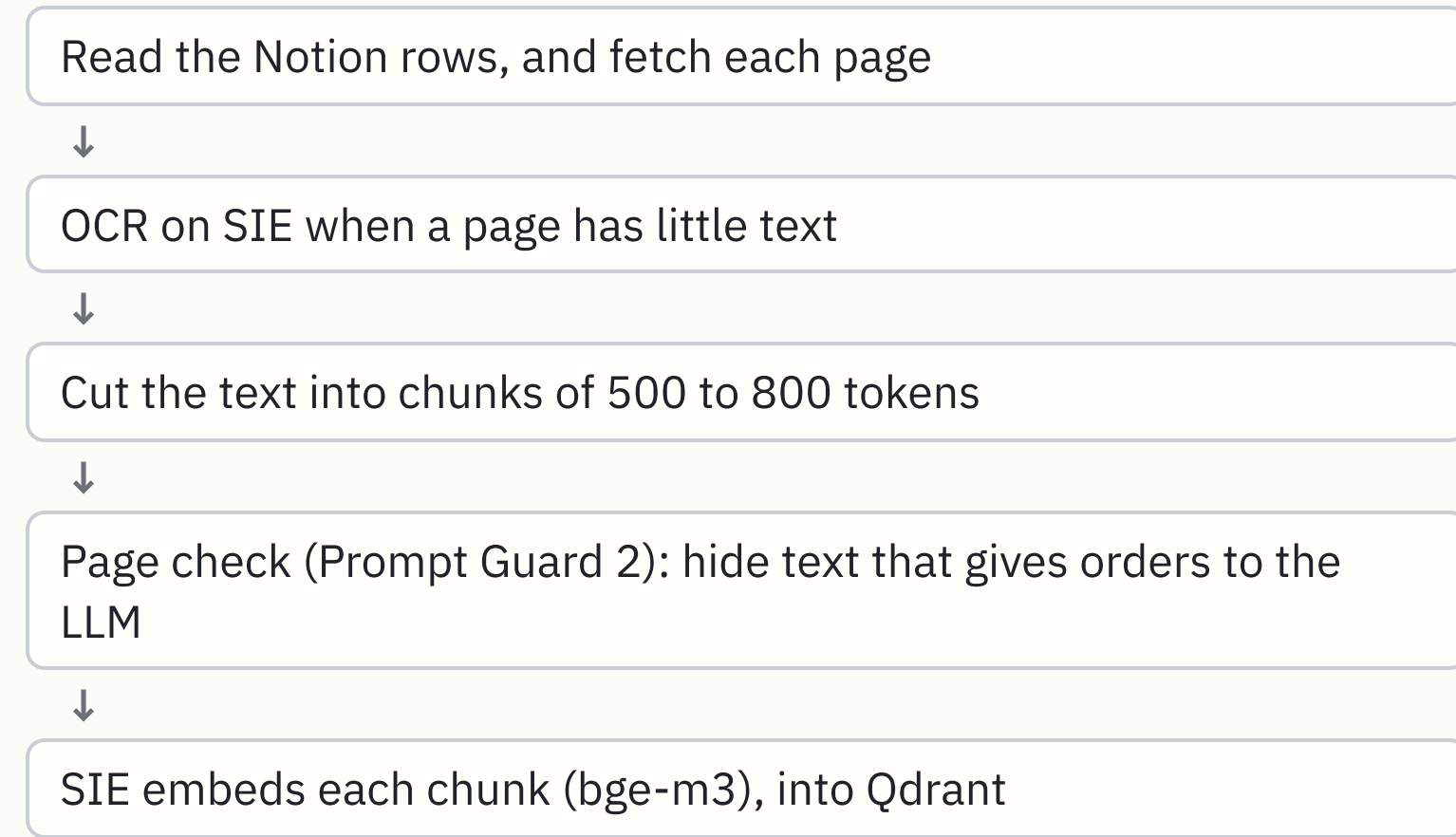
Job	Model	Where it runs
The answers and all agent steps	Gemma 4 31B, FP8 (RedHatAI), on vLLM v0.30.0	node 1: the prefill pod and the decode pod, one H100 each
Prompt injection check	Llama Prompt Guard 2, 86M	node 2: a GPU slice of 4,000 MiB
Content safety check	Nemotron 3.5 Content Safety, in NeMo Guardrails	node 2: a GPU slice of 16,000 MiB
Embeddings for the bookmark search	BAAI bge-m3	node 2: SIE, a GPU slice of 12,000 MiB
Rerank of the search results	Qwen3 Reranker 4B	node 2: SIE, the same slice
Text in figures (OCR)	LightOnOCR-2, 1B	node 2: SIE, a GPU slice of 20,000 MiB

HAMi cuts one H100 of node 2 into these slices. On the A100 node, the small models shared one GPU.

Why Gemma 4 31B
A tested FP8 checkpoint that fits one H100. It passed all 7 of our gate tests, with 97.5% correct tool calls. We dropped Qwen3.8-27B for its open vLLM bugs in the prefix cache.

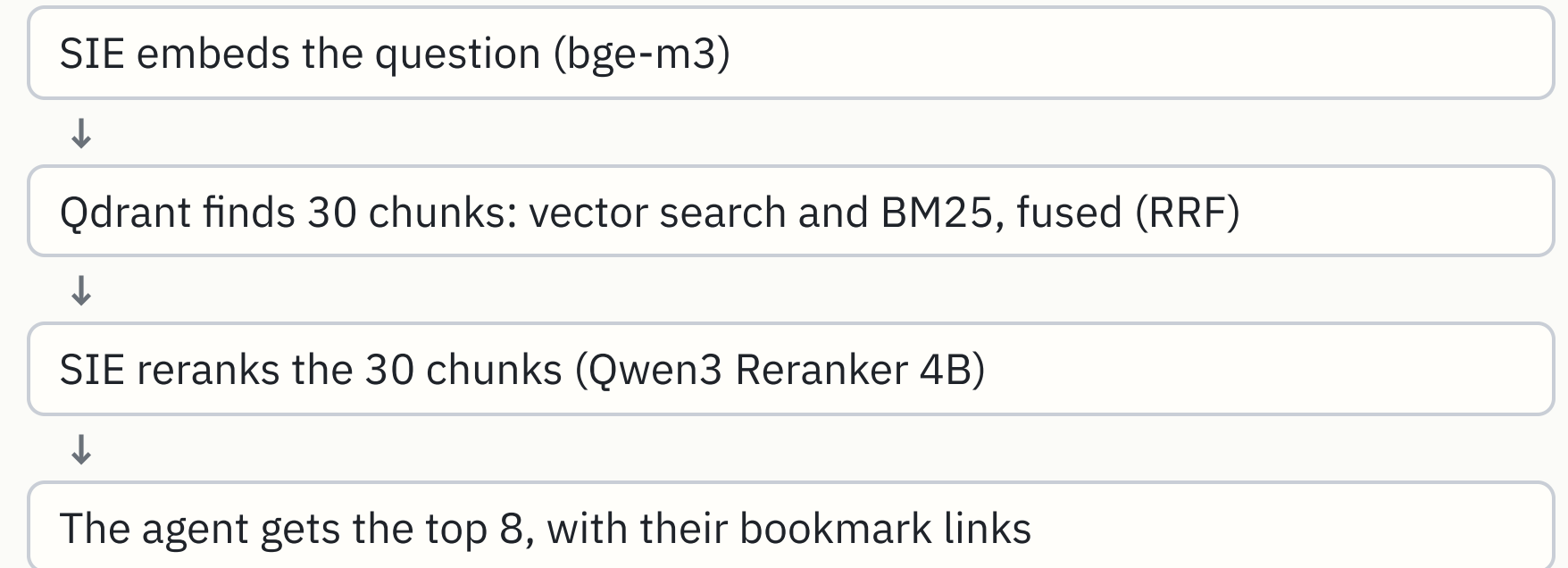
The bookmark search: ingest once, then search in each turn

Ingest: a job in the cluster, before all tests



It ran on 2026-09-28: 910 of 1,000 rows gave 4,112 chunks. Each later session restored a snapshot of Qdrant from the laptop.

Search: in each turn, a tool of the agent



SIE and Qdrant

SIE is the model server for the small models: embed, rerank, and OCR. Qdrant stores the vectors and searches them. The search makes no LLM call.

KV on paper: bytes for each token, and how many sequences fit

Gemma 4 31B, BF16 KV. Full-attention layers: 40,960 bytes for each token. Sliding-window layers: 819,200 bytes for each token, but only for the last 1,024 tokens, so 800 MiB for each sequence.

$$\text{max seqs} = (\text{HBM} - \text{weights} - \text{activations}) / \text{KV of one sequence}$$

Length	KV of one sequence	Max seqs, BF16 KV	Max seqs, FP8 KV
5,121 tokens: what the app sends (median)	0.98 GiB	36	72
8K tokens: the length that we planned	1.09 GiB	32	64
32K tokens: max_len	2.03 GiB	17	34

One H100, the KV budget after the weights: 35.3 GiB.

WHAT WE MEASURED

The KV cache that vLLM gave the decode pod (one H100).

173,657 tokens

with BF16 KV, and 345,235 tokens with FP8 KV

Model switch

We kept Gemma 4 31B, because it passed all 7 gate tests. With Muse Glimmer 30B, the KV of an 8K sequence is 83% smaller, and 189 sequences fit at 8K, not 32. We changed the KV to FP8 instead: it halves the bytes for each token.

Later in the talk: which limiter came first, and if our guess was right.

The deployment: two nodes, and an A100 fallback

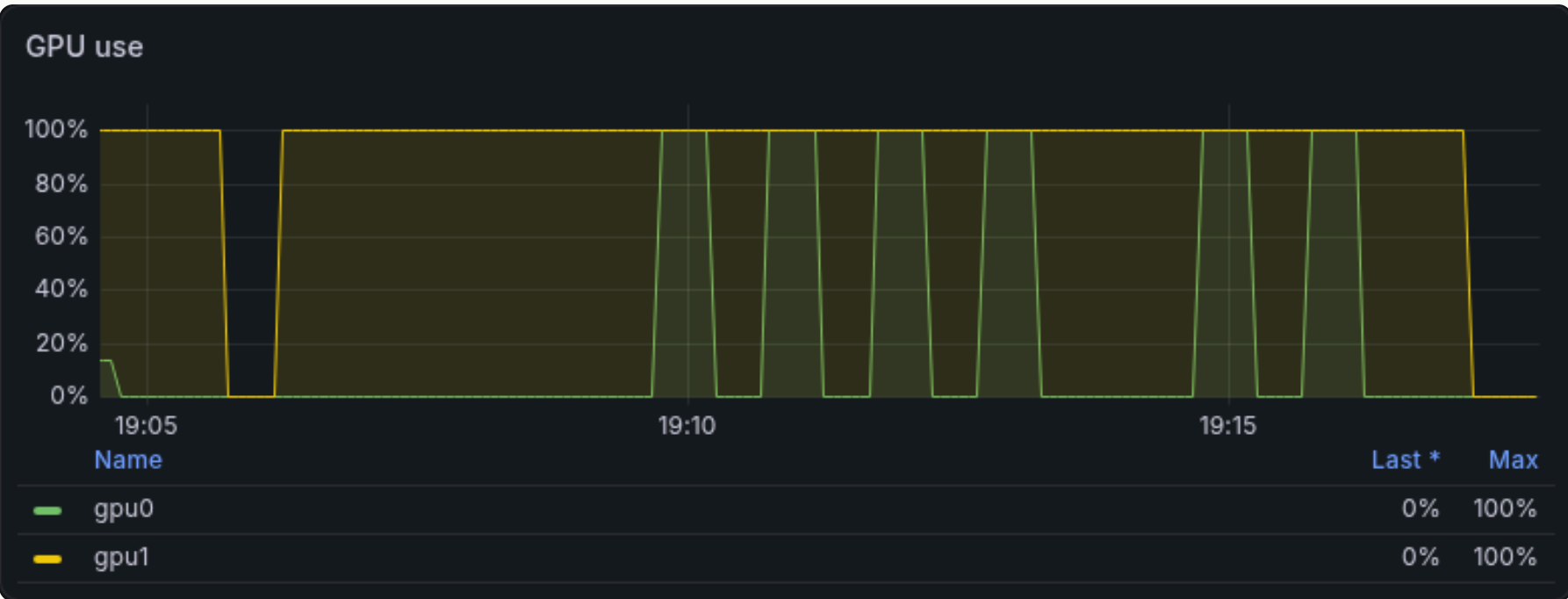
Node 1 · 2 × H100 SXM 80 GB · the engine
vllm-prefill on GPU 0 and vllm-decode on GPU 1, each on a full GPU. The LMCache server: up to 250 GiB of the 450 GiB of CPU RAM.

Node 2 · 1 × H100 80 GB · control and data
HAMi slices of one GPU for the SIE and guard models. edge, guard, Envoy, the router, the app, Qdrant, Redis, Prometheus, Grafana, and KEDA.

2026-10-01 · one node · 8 × A100 80 GB
No H100 had stock. HAMi on all GPUs: each engine pod gets a full GPU, and the node 2 pods share one GPU.

The laptop reaches the cluster only through an SSH tunnel to ports on 127.0.0.1.

Dashboard 1 · Cluster: the use of each engine GPU at 150% load. Both GPUs are at 100%.



What the app sends: short agent steps with a cached prefix

64%

of the prompt of an agent step was in the prefix cache

60%

of the calls had fewer than 2,048 new tokens

HOW WE LOAD THE CLUSTER

We replay recorded app turns at random times. A turn is one question with all its LLM calls. 100% load is 54 turns each minute, on average. At this rate, a soak test that adds load each minute refused its first call.

WHAT WE MEASURED

The tokens of 1,606 real app calls, from the Envoy logs of four capture runs.

Step	Calls	Prompt tokens (median)	Share in the prefix cache	New tokens (median)
quick	978	5,121	0.19	4,515
verify	321	1,427	0.65	133
agent	303	2,121	0.64	459

The quick steps come from the quick agent. The agent steps come from the second agent, which picks the claims and writes the final answer. The verify steps come from its helper, which checks one claim. New tokens are the prompt tokens that were not in the prefix cache, so the engine computed them.

What this means

The quick steps carry the bookmark chunks that the search found, so most of their tokens are new: the Track A shape. The agent steps share a long prefix: the Track B shape.

Guard and stay or leave happen before any GPU work

```
control/edge/guard.py:105 inspect()  
control/edge/overflow.py:19 should_leave()
```

WHAT WE MEASURED

200 chat turns through the full path: 22 prompt-injection attacks and 178 normal turns.

22 of 22 attacks blocked

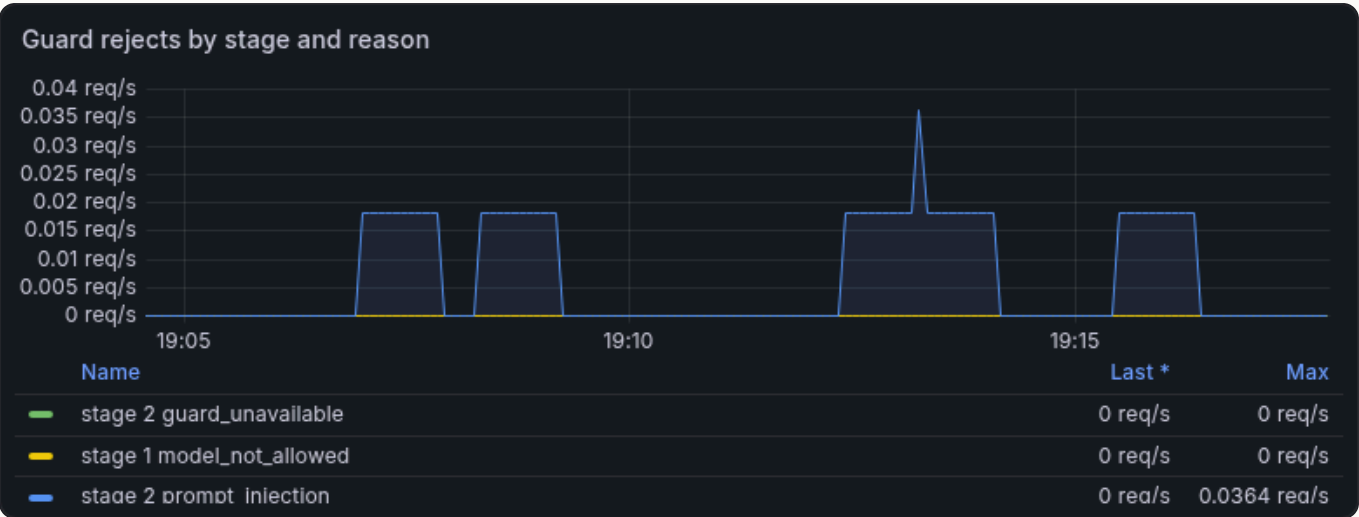
and 0 of 178 normal turns blocked. One check took 0.19 s (p50).

WHAT WE MEASURED: STAY OR LEAVE

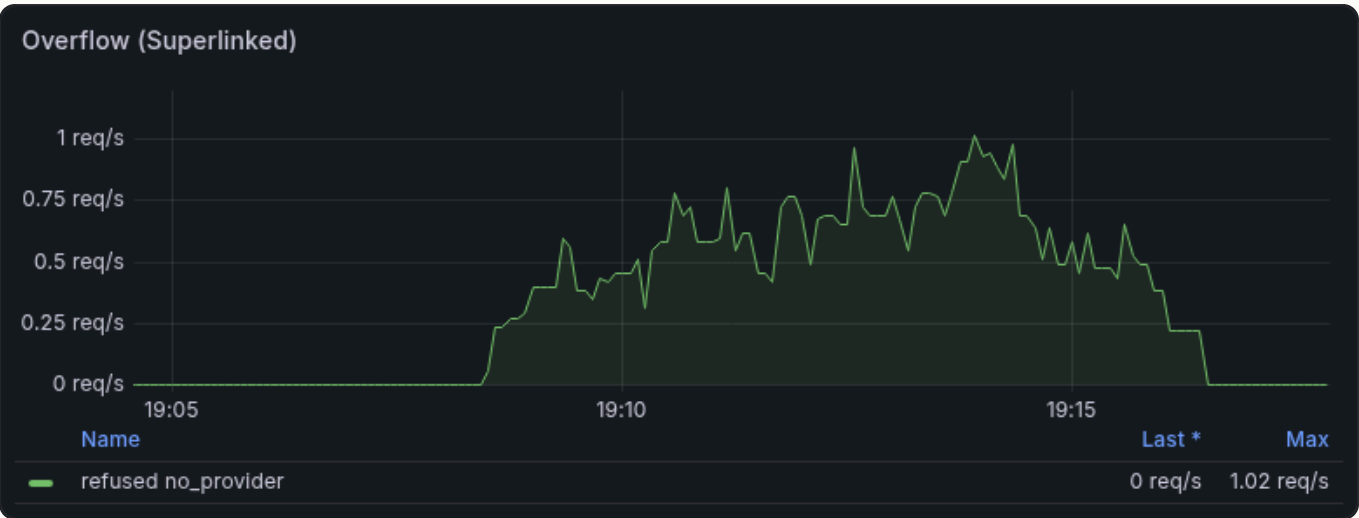
At 150% load, which calls the leave gate let go. A 429, a 500, and a batch call must stay.

The gate let only the 124 interactive 503 timeout_queue calls go. No 429 left.

Dashboard 3 · Gateway + admission: guard rejects by stage and reason



Dashboard 2 · Success and failures: the calls that may leave. The overflow was off, so they got a 503.



guard

stay or leave

admit

place

hop

warm

scale

Admit: we refuse work at the door, not in the engine

```
control/router/policy.yaml tenants,  
flow_control  
control/edge/admit.py:23 slice_oom
```

Each tenant has a token budget for each minute. At 5 queued requests or 90% KV use on a pod, llm-d stops the dispatch to that pod.

WHAT WE MEASURED

In all load tests, from 50% to 150% load: the preemptions in the vLLM engine.

0 preemptions

Before the KV was full, llm-d refused the extra work.

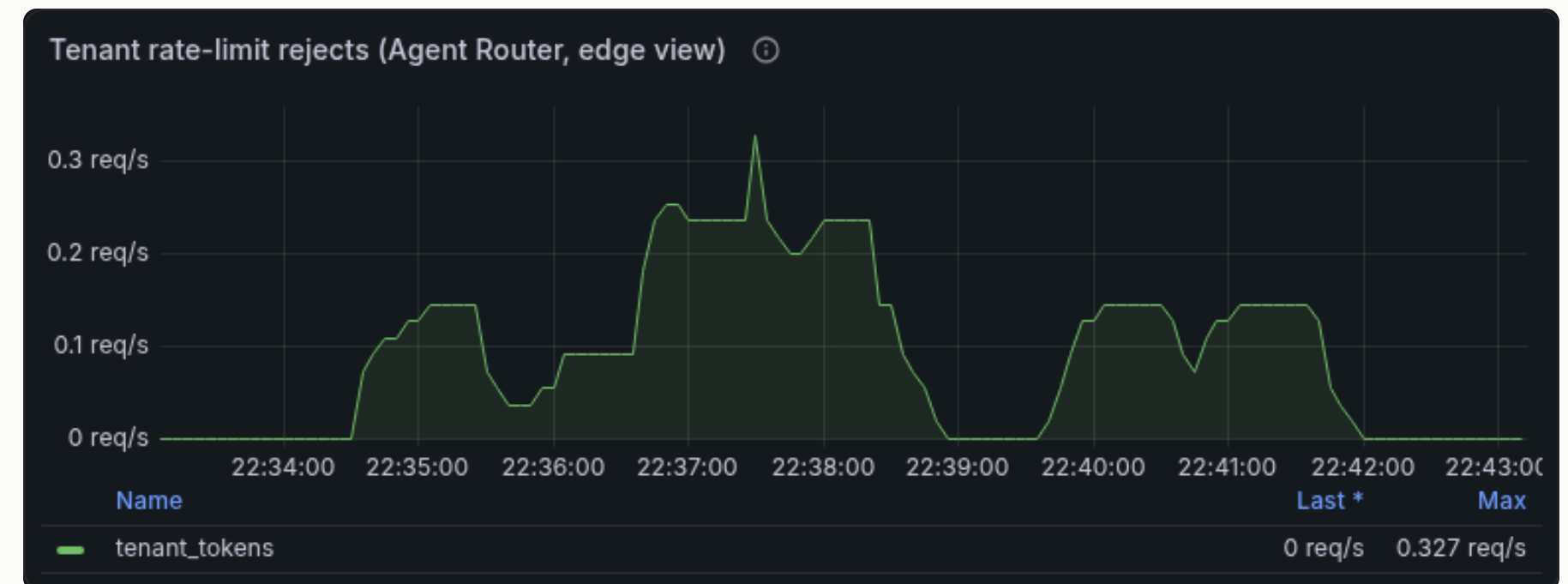
WHAT WE MEASURED

Tenant test: one noisy tenant over its token budget, at 100% load.

55 calls of the noisy tenant got 429 tenant_tokens. The other tenants kept their service.

Chetan Anand · A learning companion on a scarce GPU

Dashboard 3 · Gateway + admission: the 429 tenant_tokens rejects each second in the tenant test



Redis and the queue

Redis holds the tenant counts. The queue is in llm-d.

Place: prefix match first, then load

```
control/router/policy.yaml weights
control/router/render.py:50 render_epp_config()
```

WHAT WE MEASURED

Stale-metrics test: one of two pods sends a frozen copy of its metrics from an empty moment, at 100% load. We measured its share of the work.

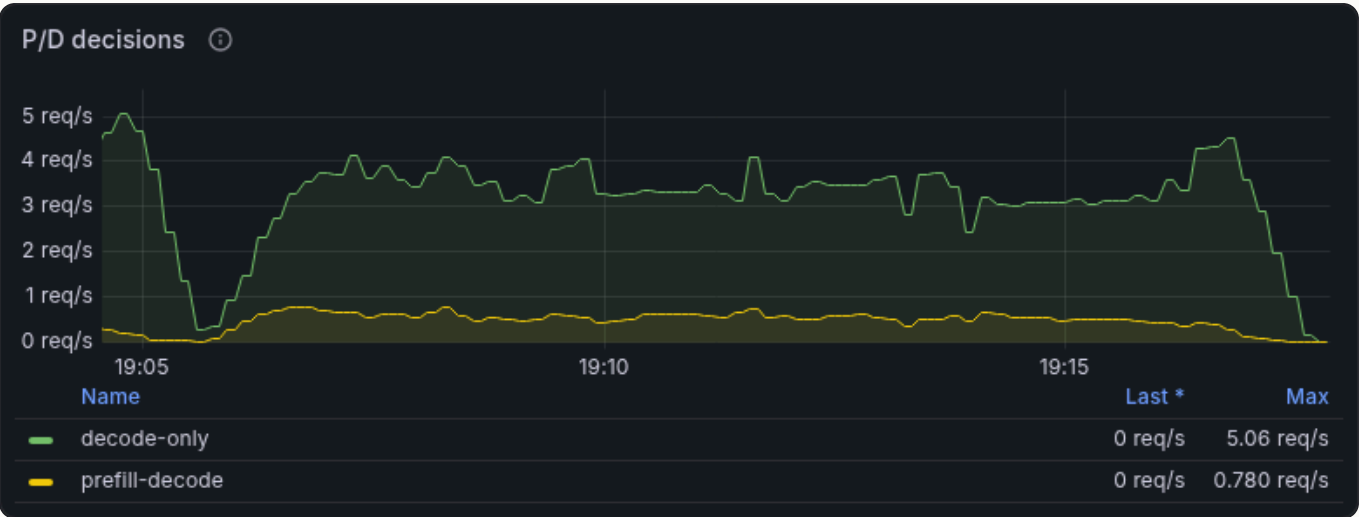
80% of the work

went to that pod. With real metrics, it got 51%.

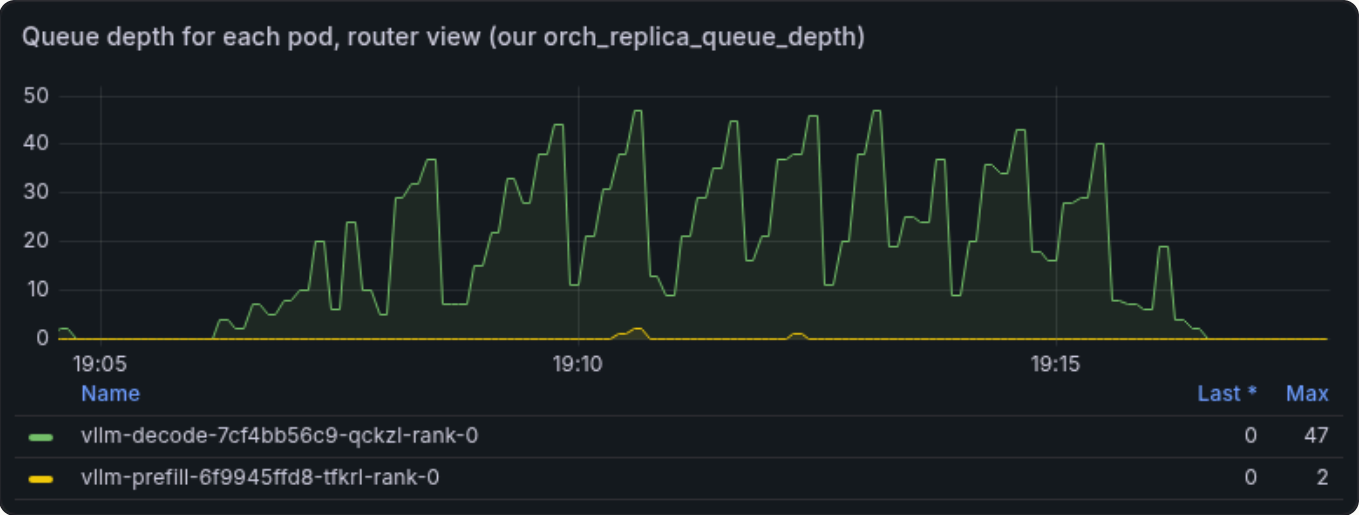
What the data changed

We cleared the prefix cache of one pod in a run. The router did not act on the clear, and all 15 warm sessions went to that pod again and missed the cache once.

Dashboard 4 · Router: the decision for each call at 150% load: decode only, or split



Dashboard 5 · Queue depth by pod: decode up to 47, prefill up to 2



guard

stay or leave

admit

place

hop

warm

scale

The hop: the KV moves through the LMCache server

```
control/router/policy.yaml
pd_non_cached_tokens: 2048
control/barrier/proxy.py:96 Barrier
tools/hop_records.py one record each hop
```

WHAT WE MEASURED

Hop test: split requests of about 9,000 tokens. The time to the first token, by the path of the KV.

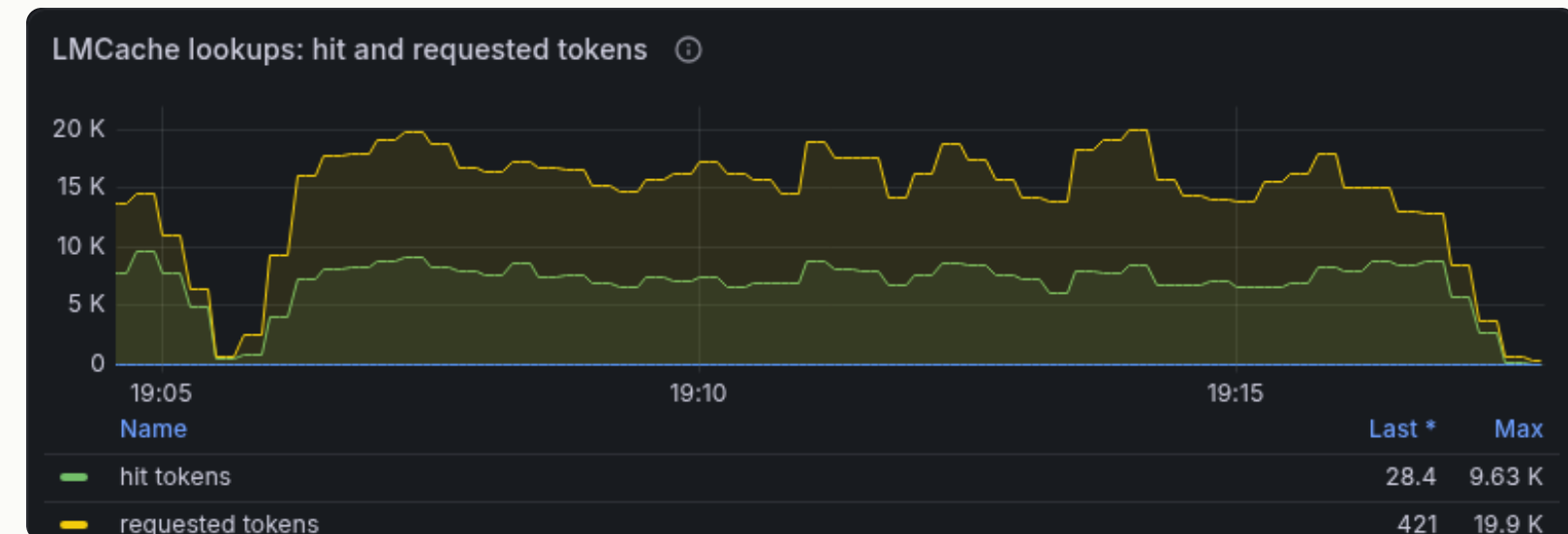
0.52 to 0.78 s

through the LMCache server, against 4.1 to 4.3 s through NIXL over TCP

What the data changed

With no store barrier, the decode pod looked for the KV before the store ended: 0 hits in 10 tries. The barrier now holds the prefill answer until the store ends.

Dashboard 7 · Hop store: the tokens that the pods ask the LMCache server for, and the hit tokens



A hop record, from the Envoy log:

```
{"src": "10.42.1.21", "dst": "10.42.1.20",
"tokens": 6624, "prefix_tokens": 6400,
"backend": "lmcache"}
```

6,400 tokens are 25 full chunks of 256. The decode pod computes the KV of only the last 224 tokens.

guard

stay or leave

admit

place

hop

warm

scale

A pod with its weights on the GPU is not warm yet

```
control/warm/logic.py:32 is_warm()  
control/warm/logic.py:36 next_ramp()
```

WHAT WE MEASURED

Restart test at 70% load: we deleted one decode pod. The TTFT p95 of the calls on the new pod, in its first minute.

7.3 s against 10.9 s

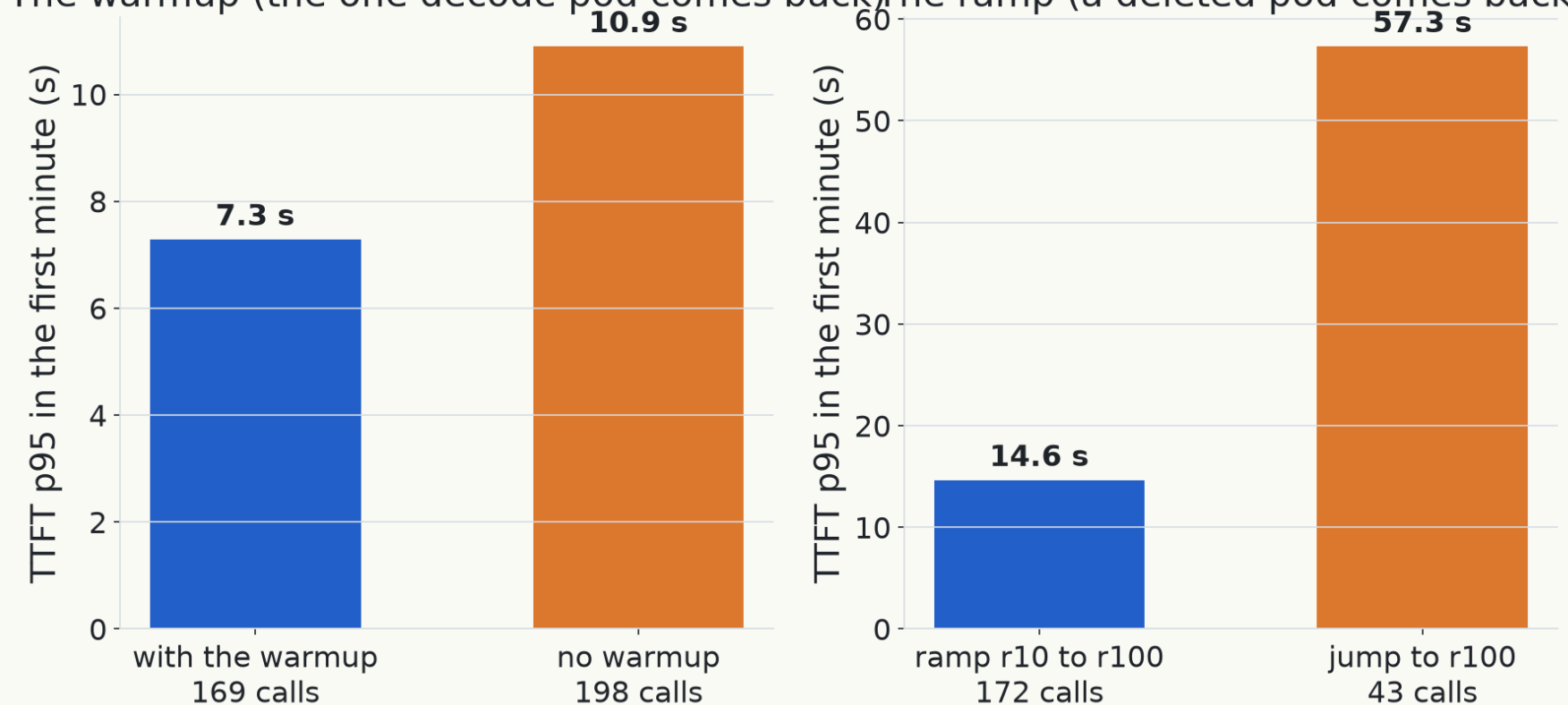
with the warmup routine, and with none. The warmup made the outage 15 s longer.

What the data changed

The ramp is a score, not a cap. In the first 10 s, the new pod got 75% of the calls.

The TTFT p95 of the calls on the new pod, in its first minute (H100)

The warmup (the one decode pod comes back) The ramp (a deleted pod comes back)



The decode pod was the limit, not prefill compute

What we expected first	Result
RAG: prefill compute	Partly. Only for long prompts.
Agents: KV blocks	Partly. The router held KV at 90%.
Not the weights	Right. 30.4 of 71.7 GiB.
Not the interconnect	Right for LMCache. Wrong for NIXL over TCP.
Not the scheduler	Wrong. The decode queue held 50 requests.

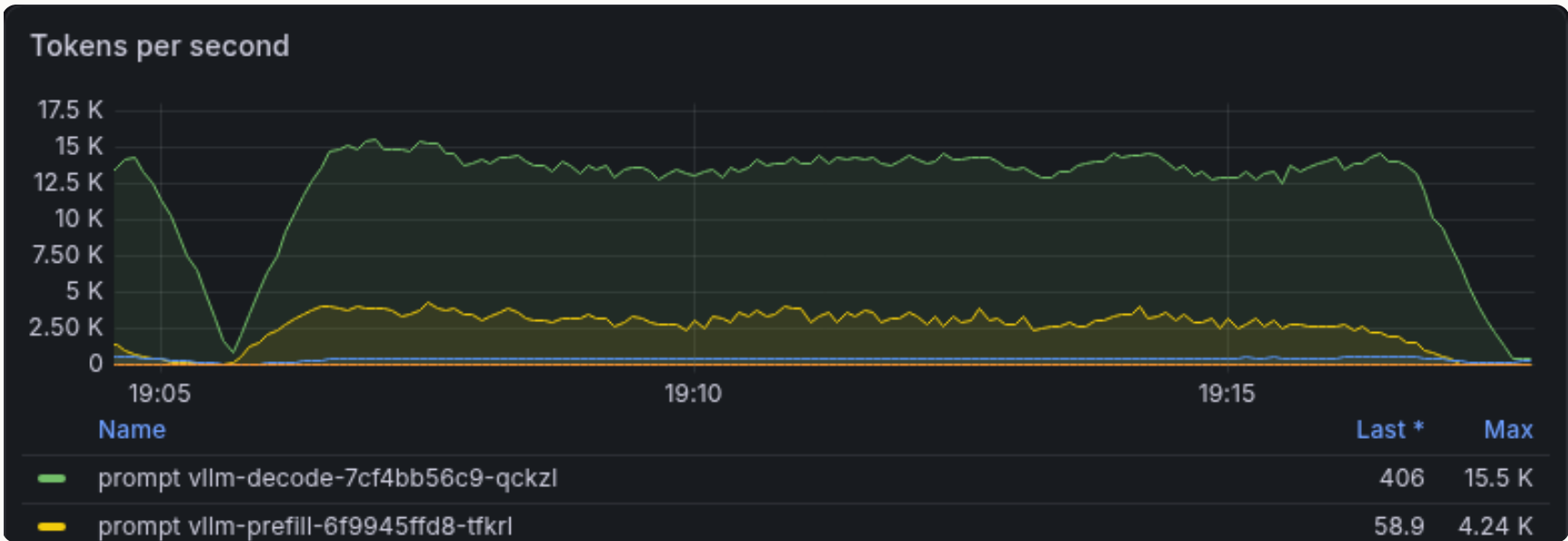
WHAT WE MEASURED

At 100% load: the prompt tokens that each pod processed each second.

16,200 against 4,550

on the decode pod and on the prefill pod

Dashboard 6 · vLLM: the prompt tokens each second for each pod, at 150% load



For our traffic, two whole pods beat a P/D split

WHAT WE MEASURED

The same recorded app traffic at three loads, on two layouts: one prefill pod and one decode pod (P/D), or two whole pods. The TTFT p50 of the interactive calls.

0.84 s against 4.59 s

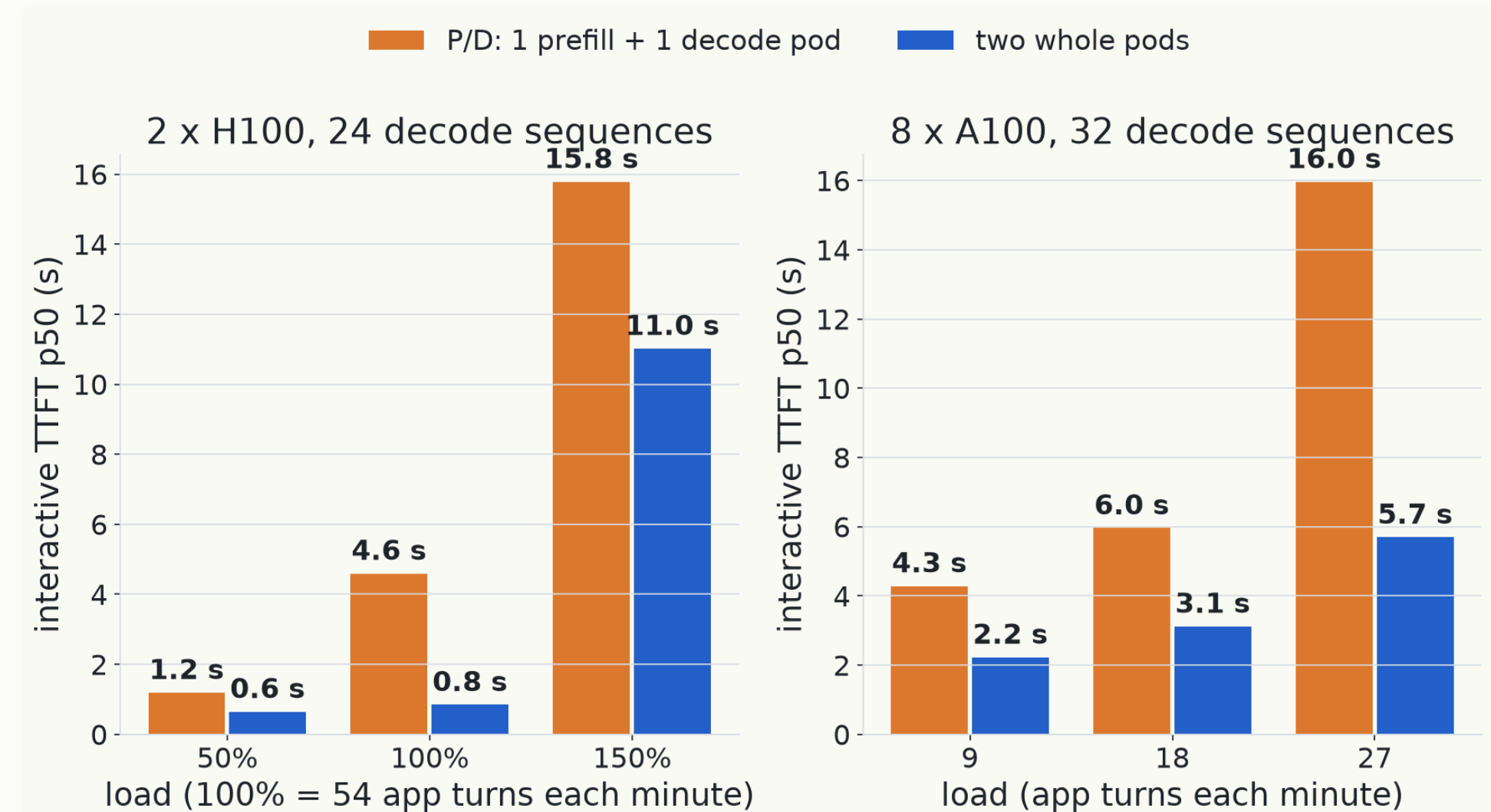
at 100% load on the H100: two whole pods against P/D. The A100 tests agree.

Why: only 10% to 13% of the calls split, so the decode pod runs the prefill of most prompts.

What the data changed

We keep the split for long uncached prompts. A P/D layout needs two or more pods in each pool, or one restart stops all split calls.

The TTFT p50 of the interactive calls, by layout and load



Scale: the planner names the pool

```
monitoring/rules.yaml:18 prefill: uncached
tokens
monitoring/rules.yaml:22 decode: running
KEDA one ScaledObject each pool
```

WHAT WE MEASURED

Scale test on 8 × A100, one pool at a time: the time from the planner request to a new pod.

15 s to the new pod

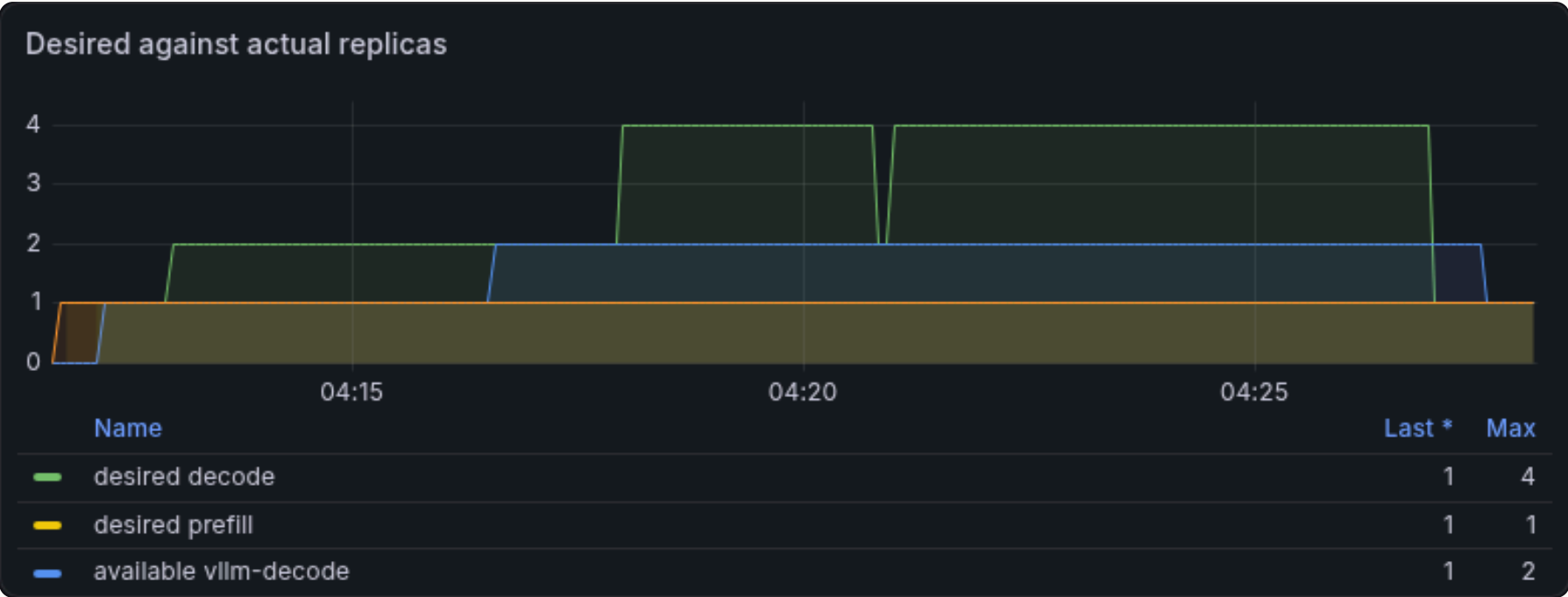
in both pools. The new pod was warm about 4 minutes later.

▶ Clip 2, 34 s, at 8 times speed: the replica panel

What the data changed

The prefill capacity value must match the GPU. With the H100 value, the planner asked for no second prefill pod on the A100.

Dashboard 8 · Pods / replicas / KEDA, in the scale test. Green: the decode pods that the planner asks for. Blue: the ready decode pods, at most 2 in this test.



What the data changed in our design

Whole pods

for our traffic. Split only long uncached prompts.

A store barrier

for the hop: completion is not visibility.

A cap for the ramp

not only a score.

GPU values

from the GPU: the planner capacity and the warm baseline.

Apply a cache clear

in the router, or each warm session misses the cache once.

10 times the traffic: more decode capacity first, 32 sequences and fp8 KV, and more RAM for LMCache. The wrong knobs: more prefill pods, longer queues, and a lower split threshold.

GPU cost: 237 USD of the 400 USD credit. The repo has the report, the notebooks, and each run.

Questions?

github.com/chetananand/learning-companion-serving

The handout questions and their evidence (1 of 2)

Question	Answer	Evidence
What is the app? Shared and unique tokens?	64% of an agent prompt is in the cache	Envoy logs
What dies at guard, admit, place, queue?	400 prompt_injection, 429 tenant_tokens, 503	docs/results.md
Where do we stop work that will time out?	band time limits: 10 s and 120 s	policy.yaml:45
Where do we protect KV?	the router stops the dispatch at 90%	policy.yaml:57
Where do we give priority to interactive?	priority bands, holdback policy	policy.yaml:47
Where do we stop one tenant?	Agent Router token windows	policy.yaml:61
Where do we hop? What is not copied?	2,048+ uncached tokens, and the last part chunk	hops.jsonl

The handout questions and their evidence (2 of 2)

Question	Answer	Evidence
Where do we evict? What becomes a ghost?	vLLM and LMCache evict, and a cleared prefix	ghost_probe.py
Where does the engine scheduler sit?	inside each vLLM pod, after our queue	manifests/engine
What limited concurrency?	the decode pod	vLLM dashboard
Four production alerts?	TTFT burn, shed rate, KV pressure, hop failures	rules.yaml
If we scale, which pool?	the pool that the planner names	rules.yaml
What changes at 10 times the traffic?	decode capacity, 32 sequences, fp8 KV	DESIGN.md
Three wrong knobs?	prefill pods, longer queues, lower split	DESIGN.md

A raw /metrics scrape of a live engine

```
# the vllm-decode pod
vllm:num_requests_running{engine="0",model_name="companion"} 1.0
vllm:kv_cache_usage_perc{engine="0",model_name="companion"} 0.01538461538461533
vllm:num_preemptions_total{engine="0",model_name="companion"} 0.0
vllm:prompt_tokens_total{engine="0",model_name="companion"} 5.426828e+06
# edge
orch_shed_total{code="400",reason="prompt_injection",stage="guard"} 7.0
orch_shed_total{code="503",reason="timeout_queue",stage="flow_control"} 138.0
orch_overflow_refused_total{why="no_provider"} 89.0
# the store barrier
barrier_hold_seconds_sum 85.915873
barrier_hold_seconds_count 171
```

Saved from the live A100 node before we stopped it. The report has more scrapes.

Faults that we found and fixed

Fault	What happened	Fix
hop race	The decode lookup ran before LMCache ended the store.	the store barrier
Envoy buffer	Envoy gave 413 for prompts of about 12K tokens.	a 4 MiB buffer
gateway timeout	The AI Gateway stopped each request at 60 s.	a timeout of 300 s
lost pod label	The warm-controller series lost the pod label.	honorLabels: true
prefill restart	The one prefill engine restarted under overload.	not fixed: two pods
cache reset	The cache reset said 200 with success false.	a forced reset
hop records	The hop records named NIXL for each run.	the real connector

The demo questions: 8 of 12 on the H100, 6 of 12 on the A100

Run	Passed	What failed
H100, 2026-09-30	8 of 12	4 questions in the fact check
A100, 2026-10-01	6 of 12	5 questions in the fact check, and 1 slow first token
After the fixes	the OCR question	Claim checks still time out or find no evidence.

The failures are in the fact check, not in the serve path.

The queue questions, with the notebook answers

Question	Answer
Our queue or the vLLM queue?	ours before the pick (up to 48 and 24), vLLM after (up to 50)
Waiting, running, preempted?	decode: 24 running and 50 waiting, and 0 preemptions
Queue depth for each pod?	6 or fewer for most traffic, and up to 36 for the mixed traffic
A 32K retrieve and an agent step?	the agent step goes first, in all rounds
PagedAttention or prefix cache?	the prefix cache saved prefill work, and fp8 KV saved memory
Engine flags?	decode: 24 sequences, chunks of 2,560. Prefill: 8, chunks of 16,384.
KV full after admit?	we refuse at the door, and no pod preempted
Client gone?	vLLM abort_requests frees the KV
A worker returns?	a ramp while the p99 holds

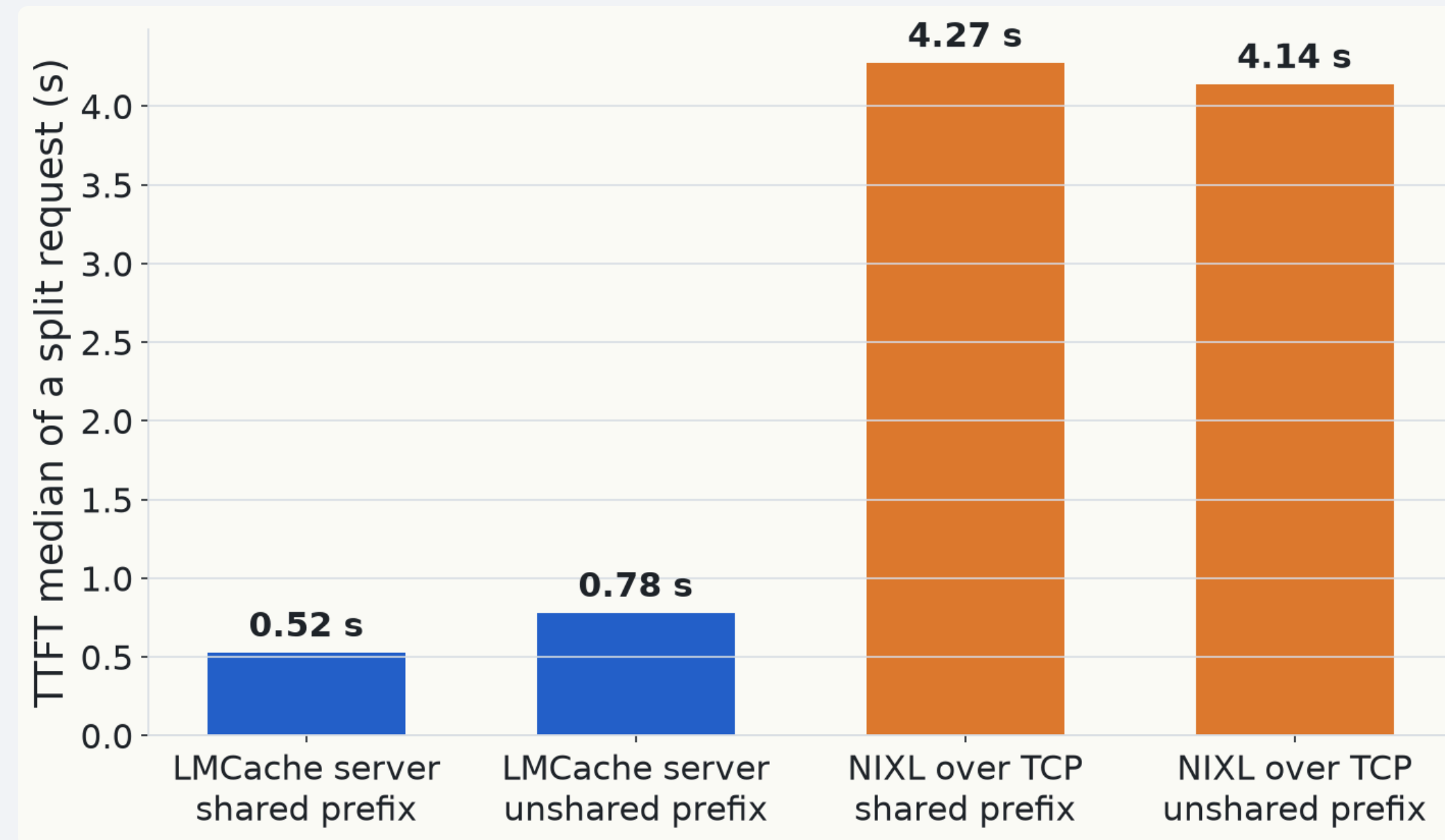
The cost of each GPU block

Date (PDT)	Shape	Hours	USD
2026-09-27	2 × H100 SXM (first engine test)	0.37	3.07
2026-09-28	2 × A6000 (first platform test)	1.92	4.17
2026-09-29	1 × H100 SXM (node 2)	11.40	48.91
2026-09-29	2 × H100 SXM (node 1)	9.85	82.56
2026-09-30	4 × H100 SXM (no node 2 came)	0.75	12.20
2026-10-01	8 × A100 80 GB (one node)	3.88	86.51
Total	of the 400 USD credit		237.43

Appendix

The hop: the LMCache server against NIXL

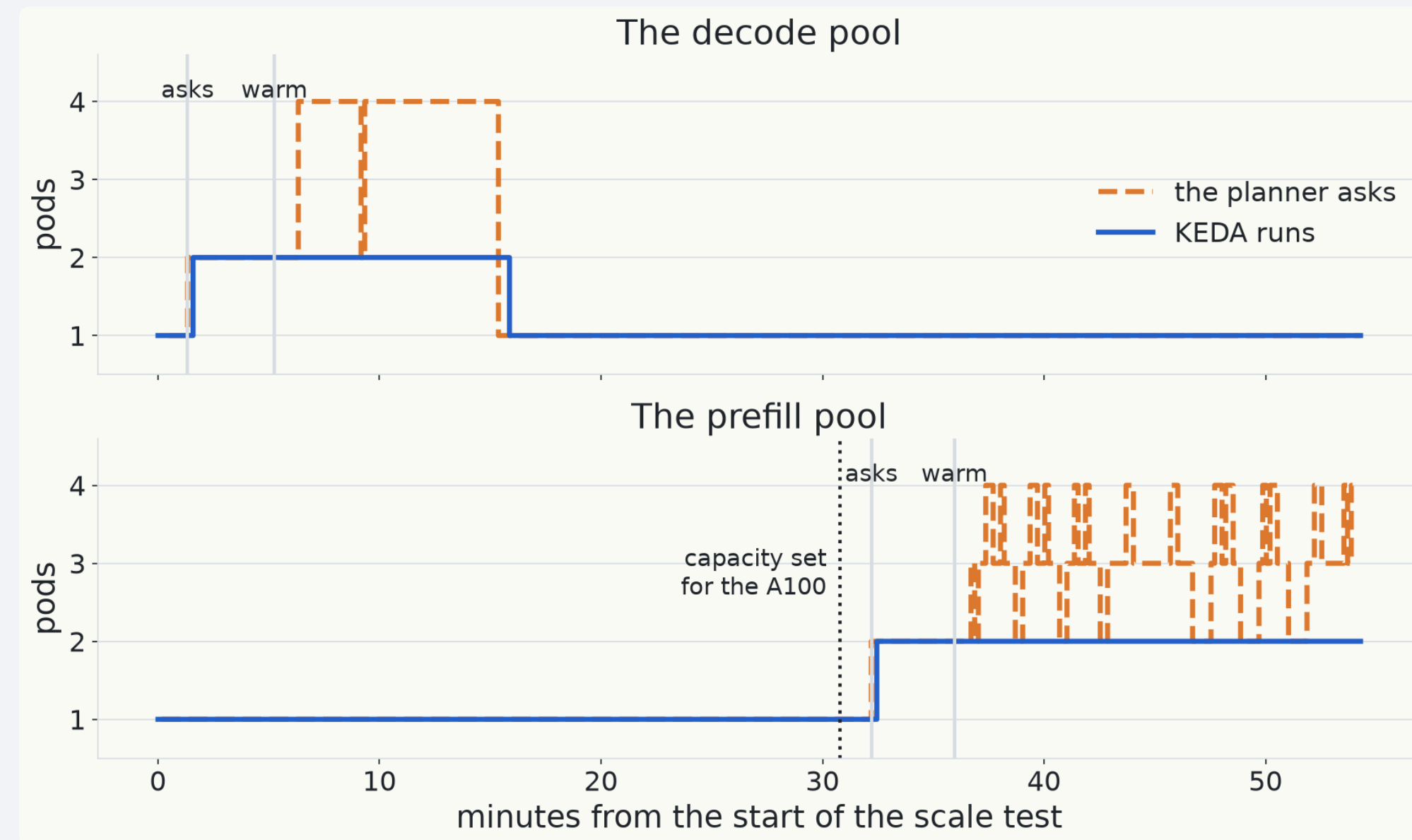
The time to the first token of a split request of about 9,000 tokens



Appendix

The scale test: the planner against KEDA, in each pool

The pods that the planner asks for, and the pods that KEDA runs. One pool at a time.



The hop at production scale: what we keep, what we change

Keep

The router decides, and the sidecar runs the two steps. The one-token prefill request: the smallest vLLM request that runs a full prefill.

WHAT WE MEASURED

The store barrier on the A100 node, under load. It waits for every store that LMCache had at that time, with a cap of 0.5 s.

When	Holds	Hit the 0.5 s cap
P/D load tests	237	170 (72%)
End of session, prefill pod 1	171	163 (95%)
End of session, prefill pod 2	68	67 (99%)
Start of session, one request at a time	3	0 (0.19 s on average)

A production hop needs

1. A store signal for each request, in place of the global store counters.
2. RDMA between nodes, with NIXL or a distributed KV store. Our LMCache path works only inside one node, and NIXL over TCP took about 4 s.
3. Two or more pods in each pool, and a fallback: if the prefill request fails, the decode pod does the prefill itself.
4. mTLS between pods. Our sidecar ran with plain HTTP.
5. P/D only where long uncached prompts are common. For our traffic, two whole pods were faster.